# MULTI-CORE PROGRAMMING

## Exam Notes: Chapter 3

### Fundamental Concepts of Parallel Programming

**Source:** *Multi-Core Programming: Increasing Performance through Software Multi-threading*, Shameem Akhter and Jason Roberts, Intel Press, 2006. Chapter 3, pp. 37 to 52.

## How to use these notes

- All content is taken from Chapter 3 of the textbook only. Same colour code as the Chapter 1-2 notes: yellow = must-remember, green = worked example/formula, red = trap.
- Diagrams are redrawn from Figures 3.1, 3.3, 3.4 and 3.5. The gardener diagram summarises the book's analogy (not a book figure).
- Part A: notes. Part B: quick revision sheet. Part C: practice questions with model answers.

## Contents


| Part | Content |
|---|---|
| A | 3.1 Designing for threads; 3.2 Challenges; 3.3 Patterns; 3.4 Error diffusion case study; 3.5 Key points |
| B | Quick revision sheet |
| C | Practice questions |

# PART A: CHAPTER 3 - Fundamental Concepts of Parallel Programming

**Chapter idea:** Parallel programming centres on the design, development and deployment of threads within an application and the coordination between threads and their operations. The chapter shows how to break programming tasks into chunks suitable for threading, then applies this to the apparently serial problem of **error diffusion**.

## 3.1 Designing for threads

- Traditional models (e.g. object-oriented programming): a program begins at a defined point such as **main()** and works through tasks in succession. If it relies on user interaction, the main processing instrument is a **loop in which user events are handled**; each event (e.g. a button click) triggers an established sequence of actions ending with a wait for the next event.
- This is a simple world: **only one thing happens at any moment**. Any required order is imposed by the developer; each step flows into the next to a predictable conclusion.
- To move to a parallel model, designers must **rethink process flow**: instead of a sequential sequence, identify activities that can run in parallel. See the program as a **set of tasks with dependencies** between them.

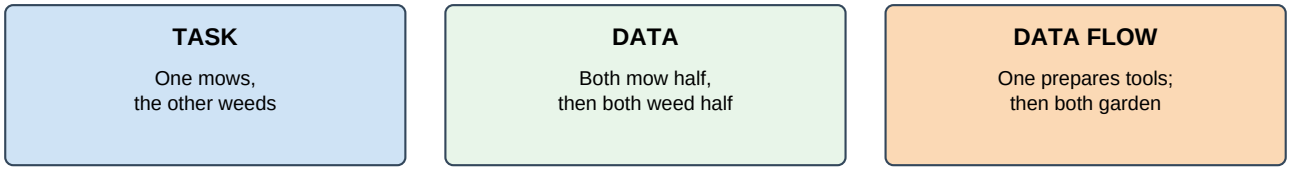
**DEFINITION: Decomposition**

- Breaking a program into individual tasks and identifying the dependencies between them is called **decomposition**.
- A problem may be decomposed **by task, by data, or by data flow**.

### Table 3.1: Major forms of decomposition

| Decomposition | Design | Comments |
|---|---|---|
| Task | Different activities assigned to different threads | Common in GUI apps |
| Data | Multiple threads performing the same operation but on different blocks of data | Common in audio processing, imaging and scientific programming |
| Data flow | One thread's output is the input to a second thread | Special care needed to eliminate startup and shutdown latencies |

**Gardener analogy for the three decompositions**




*The book's gardening analogy for the three decompositions*

## Task decomposition

- Decomposing a program **by the functions it performs**. One of the simplest ways to achieve parallel execution.
- Individual tasks are catalogued; if two can run concurrently, the developer schedules them to do so. Usually needs **slight modifications to the individual functions** to avoid conflicts and to indicate that the tasks are no longer sequential.
- Gardening: one gardener mows while the other weeds. They need **coordination** so the weeder is not sitting in the middle of a lawn that needs mowing.
- **Program example: word processor (e.g. Microsoft Word).** While the user types into a very long document, **pagination happens in the background** (page count in the status bar rises). Text entry and pagination are two separate tasks broken out by function. Otherwise the user would have to wait for full pagination before typing, as on early PC word processors.

## Data decomposition (data-level parallelism)

- Breaks down tasks **by the data they work on**, not by the nature of the task. Many threads perform **the same work on different data items**.
- Example: recalculating a large spreadsheet. Instead of one thread doing all calculations, use two threads each doing half, or **n threads each doing 1/n of the work**.
- Gardening: both mow half the property, then both weed half the flower beds.
- Which form is better depends on **system constraints**. If the lawn is so small that it does not need two mowers, task decomposition is better (one gardener mows); data decomposition can then be used when both start weeding in parallel.
- **Scalability benefit:** as the number of cores increases, data decomposition allows the **problem size to be increased**, so more work is done in the same time. Example: two more gardeners are assigned to a second, equal-sized yard, doubling the work done in the same time. (Links to Gustafson's Law, Ch. 1.)

## Data flow decomposition

- Used when the critical issue is not what tasks do the work but **how data flows between tasks**.
- Classic example: the **producer/consumer problem**. The output of one task (producer) becomes the input of another (consumer). They run in different threads, and the consumer **cannot start until the producer finishes some portion of its work**.
- Gardening: one gardener prepares tools (fuels the mower, cleans shears) for both. No gardening can occur until this is mostly finished; the delay pauses the second task, after which both continue in parallel.
- Typical scenarios: programs that **read a file** (I/O result feeds a possibly threaded next step, which cannot begin until reading is complete or has progressed enough) and **parsing** (input must be parsed/analysed semantically before back-end work such as **code generation in a compiler**).

**EXAM FAVOURITE: Dimensions of the producer/consumer problem**

- **Dependence delay:** the dependence between consumer and producer can cause significant delays if implemented badly. A performance-sensitive design understands the exact nature of the dependence and diminishes the delay; it avoids consumer threads idling while waiting for producers.
- **Clean hand-off:** ideal case (e.g. the file parser) is a completely clean hand-off: output is context-independent and the consumer needs to know nothing about the producer. Often there is no such clean division and scheduling their interaction needs careful planning.
- **Load imbalance:** if the consumer is finishing while the producer is completely done (or vice versa), one thread sits idle while others work. This violates the objective of **balancing loads so all available threads stay busy**, and is hard to avoid because of the logical relationship between the threads.

## Implications of different decompositions

- Goal = **ease of programming** and tasks neatly partitioned by functionality: **task decomposition** is more often than not the winner.
- **Data decomposition** adds code-level complexity, so reserve it for when data is easily divided and **performance is important**.
- The most common reason for threading is performance, and then the choice is harder. Often the **problem domain dictates** the choice; some problems have no clear bias.
- Example: processing images in a **video stream** with no dependency between frames. Task decomposition: one thread decodes, another colour-balances, etc. Data decomposition: each thread does all the work on one frame, then moves to the next.
- Gardening version: two gardeners, two lawns, two flower beds. Should one only mow (task) or should both mow together, then weed together (data)? A **resource constraint** (only one mower) settles it quickly. If each has a mower, only careful analysis tells. Here task decomposition looks better because the **start-up time for mowing is saved** if only one mower is used.

**Remember**

- The right answer comes from **careful planning and testing**. **Empirical timing and evaluation** play a more significant role in design choices in parallel programming than in standard single-threaded programming.

## 3.2 Challenges you'll face

Threads improve performance by letting two or more activities occur simultaneously, but add complexity because more than one activity is occurring. Managing simultaneous activities and their possible interaction leads to **four types of problems**:

| Challenge | Meaning (from the book) |
|---|---|
| Synchronization | The process by which two or more threads coordinate their activities. E.g. one thread waits for another to finish a task before continuing. |
| Communication | The bandwidth and latency issues associated with exchanging data between threads. |
| Load balancing | Distribution of work across multiple threads so they all perform roughly the same amount of work. |
| Scalability | Making efficient use of a larger number of threads when software runs on more capable systems. E.g. a program that uses four cores well: does it scale properly on eight cores? |

- Each must be handled carefully to maximise performance; later chapters describe how to address them on multi-core systems.

## 3.3 Parallel programming patterns

- Just as object-oriented programmers use **design patterns**, parallel programming problems generally fall into one of several well-known patterns.

### Table 3.2: Patterns and their decomposition

| Pattern | Decomposition |
|---|---|
| Task-level parallelism | Task |
| Divide and Conquer | Task / Data |
| Geometric Decomposition | Data |
| Pipeline | Data flow |
| Wavefront | Data flow |

### Task-level parallelism pattern

- Focus directly on the tasks. The problem is decomposed into a set of tasks that **operate independently**. Often necessary to **remove dependencies** between tasks or **separate dependencies using replication**.
- Fits **embarrassingly parallel** problems (no dependencies between threads) and **replicated data** problems (dependencies between threads can be removed from the individual threads).

### Divide and conquer pattern

- Problem is divided into a number of **parallel sub-problems**; each is solved independently; results are **aggregated** into the final solution. Because sub-problems are independent, they can run in parallel.
- Widely used in sequential algorithms such as **merge sort**, which are very easy to parallelise. Typically gives **good load balancing** and **good locality** (important for effective cache usage).

### Geometric decomposition pattern

- Based on parallelising the **data structures** used in the problem. Each thread is responsible for operating on data **'chunks'**. Applied to problems such as **heat flow and wave propagation**.

### Pipeline pattern

- Identical in idea to an **assembly line**. Break the computation into a **series of stages** and have each thread work on a different stage simultaneously. (This is the scalable solution to the producer/consumer problem.)

### Wavefront pattern

- Useful when processing data elements **along a diagonal in a two-dimensional grid** (Figure 3.1).
- Numbers show processing order. Elements on diagonal '3' depend on elements '1' and '2' having been processed. Shaded elements = already processed.
- Critical: **minimise idle time** of each thread. **Load balancing is the key to success** with this pattern.

**Wavefront data access pattern (Figure 3.1)**

| | | | | | |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 |
| 2 | 3 | 4 | 5 | 6 | 7 |
| 3 | 4 | 5 | 6 | 7 | 8 |
| 4 | 5 | 6 | 7 | 8 | 9 |
| 5 | 6 | 7 | 8 | 9 | 10 |
| 6 | 7 | 8 | 9 | 10 | 11 |

Shaded = already processed
Yellow = diagonal being processed (3)
Number = processing order;
diagonal '3' needs '1' and '2' first

*Figure 3.1 (redrawn): wavefront data access pattern*

- Further reading cited by the book: *Patterns for Parallel Programming* (Mattson 2004).

## 3.4 A motivating problem: error diffusion

### What it is

- Used in many computer graphics and image processing programs. Originally proposed by **Floyd and Steinberg (1975)**.
- A technique for **displaying continuous-tone digital images on devices with limited colour (tone) range**. Problem: printing an **8-bit grayscale** image on a **black-and-white (bi-level)** printer, which cannot print 8-bit natively, so it must simulate shades of gray by an approximation technique.
- Output image (Figure 3.2) consists of pixels of only two colours, black and white; at normal print resolution it looks similar to the original, but zoomed to 400% the 2-level rendering is clearly visible.

### The basic three-step algorithm

| Step | What happens |
|---|---|
| 1. Output value | Determine the output for the current pixel using **quantization** or, in the binary case, **thresholding**. For 8-bit input on a 1-bit device: input in **[0,127] gives 0**; input in **[128,255] gives 1**. |
| 2. Error | Compute the **error** between what should be displayed and what actually is. Normalise the output to the input scale first: output 0 stays 0, output 1 becomes 255. Error = input - normalised output. |
| 3. Distribute | Distribute the error on a **fractional basis to neighbouring pixels** (Figure 3.3). |

**WORKED EXAMPLE (from the book)**

- Input pixel = **168**. It is above the threshold (128), so output = **1** (stored in the output array).
- Normalised output = 255. Display error = 168 - 255 = **-87**.

- **Floyd-Steinberg weights: 7/16** of the error goes to the pixel to the **right**; **5/16** to the pixel in the next row **directly below**; the remainder (**3/16** below-left, **1/16** below-right) propagates similarly. Other weighting schemes exist but all error diffusion algorithms follow this general method. The three steps are applied to **all pixels**.

### Listing 3.1: serial C implementation (key idea)

```
void error_diffusion(unsigned int width, unsigned int height,
                     unsigned short **InputImage, unsigned short **OutputImage)
{
  for (unsigned int i = 0; i < height; i++)
    for (unsigned int j = 0; j < width; j++)
    {
      /* 1. Compute the value of the output pixel */
      if (InputImage[i][j] < 128) OutputImage[i][j] = 0;
      else                        OutputImage[i][j] = 1;
      /* 2. Compute the error value */
      int err = InputImage[i][j] - 255*OutputImage[i][j];
      /* 3. Distribute the error */
      InputImage[i][j+1]   += err * 7/16;
      InputImage[i+1][j-1] += err * 3/16;
      InputImage[i+1][j]   += err * 5/16;
      InputImage[i+1][j+1] += err * 1/16;
    }
}
```

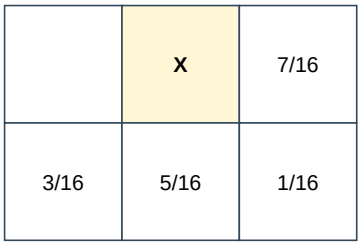
### Analysis: why it looks serial

- At first glance it is **inherently serial**: errors are distributed to neighbours as pixels are computed, so the **previous pixel's error must be known** before the next pixel's value can be computed. This interdependency implies one pixel at a time. But with a different view it can be made suitable for multithreading.

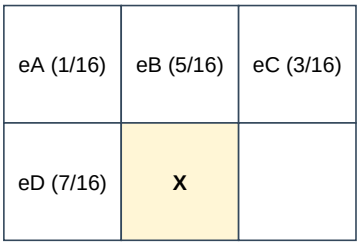
### Alternate approach: parallel error diffusion

- Consider the receiving pixel's perspective (Figure 3.4). A pixel cannot be processed until its **spatial predecessors** are done, so there are **multiple producers** (error values) and a **consumer** (the current pixel). The flow of error data is critical, so the problem breaks down by **data-flow decomposition**.

**Error distribution (Fig 3.3) vs receiving-pixel view (Fig 3.4)**




(right, below-left, below, below-right)




3 values from previous row + 1 from left (current row)

*Figures 3.3 and 3.4 (redrawn): error spreading from X, and errors received by X (eA, eB, eC from previous row; eD from the left)*

- **Bad partition:** one thread does even pixels of a row and another does odd pixels. **Ineffective**: each thread blocks waiting for the other, and performance could be **worse than sequential**.
- To split work, **reduce or eliminate dependency between pixels**. To process a pixel it needs **three error values (eA, eB, eC) from the previous row** and **one from the pixel immediately left** in the current row. (Footnote: eA = eD = 0 at the left edge, column 0; eC = 0 at the right edge, column W-1, W = pixels per row.)
- **Solution: each thread processes one row.** Once a row has processed its first few pixels, the thread responsible for the next row can begin. Multiple threads process multiple rows simultaneously (Figure 3.5). This is the **wavefront pattern**.

**Parallel error diffusion: one thread per row (Figure 3.5)**

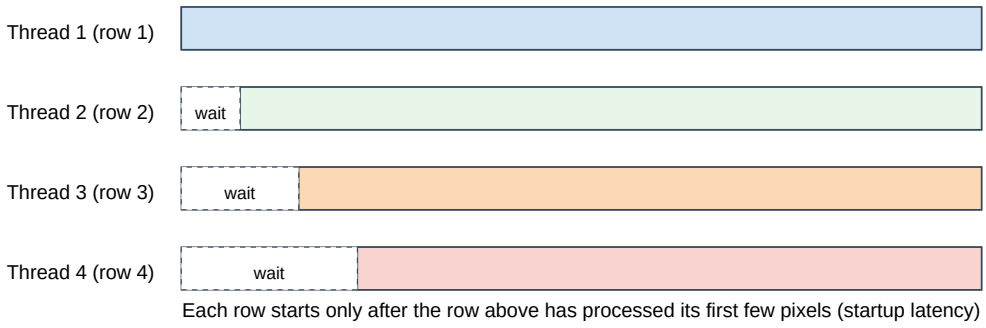



*Figure 3.5 (schematic redraw): multi-thread, multi-row error diffusion; each thread starts a little after the one above*

- A small **latency at the start of each row** exists because the previous row's error data must be computed first. Such latency is generally unavoidable in producer-consumer implementations, but its impact can be minimised by proper workload partitioning.
- Here the latency is **two pixels** before the next thread can start. An **8.5 x 11 inch page at 1,200 dpi has 10,200 pixels per row**, so a two-pixel latency is **insignificant**.

### Other alternatives: page-level and hybrid

- Instinct is to look for independent tasks. Simplest idea: **process each page separately** (each page is an independent data set). The book rejects this for three reasons:
  - An image may **span multiple pages**; this would impose a restriction of **one image per page**.
  - **Increased memory**: an 8.5 x 11 inch page at 1,200 dpi consumes **131 MB of RAM**; intermediate results must be saved, so it is less memory efficient.
  - A common use case prints **only a single page at a time**; page-level subdivision gives **no improvement over sequential**.
- **Hybrid approach:** subdivide pages and process regions of a page in a thread (Figure 3.6). Each thread works on sections from **different pages**, which increases startup latency: Thread 2 has a **1/3 page** startup latency, Thread 3 has **2/3 page**. Somewhat improved but suffers similar limitations to page-based partitioning.

**CONCLUSION**

- To avoid these limitations, use the **row-based** error diffusion implementation of Figure 3.5.

**Common mistakes / traps**

- Calling error diffusion 'inherently serial': the book shows a **simple transformation** makes it parallel.
- Splitting a row into even/odd pixels: threads block on each other; can be slower than serial.
- Confusing decompositions: **task** = different activities; **data** = same operation on different data; **data flow** = one thread's output feeds another.
- Forgetting that data decomposition lets **problem size grow** with core count.

## 3.5 Key points (as given in the book)

- Decompositions fall into one of three categories: **task, data and data flow**.
- **Task-level parallelism** partitions work between threads based on tasks.
- **Data decomposition** breaks down tasks based on the data the threads work on.
- **Data flow decomposition** breaks down the problem in terms of how data flows between the tasks.
- Most parallel programming problems fall into one of several well-known patterns.
- The constraints of **synchronization, communication, load balancing and scalability** must be dealt with to get the most benefit from a parallel program.
- Many problems that appear serial may, through a **simple transformation**, be adapted to a parallel implementation.

# PART B: QUICK REVISION SHEET

## One-line definitions

| Term | Definition |
|---|---|
| Decomposition | Breaking a program into tasks and identifying dependencies between them |
| Task decomposition | Split by the functions performed; different activities in different threads |
| Data decomposition (data-level parallelism) | Same work on different blocks of data; n threads each do 1/n of the work |
| Data flow decomposition | Split by how data flows between tasks; one thread's output is another's input |
| Producer/consumer | Consumer cannot start until producer finishes some portion of its work |
| Synchronization | Threads coordinating their activities (e.g. waiting for another to finish) |
| Communication | Bandwidth and latency issues of exchanging data between threads |
| Load balancing | Distributing work so all threads do roughly equal work |
| Scalability | Making efficient use of more threads on more capable systems |
| Embarrassingly parallel | No dependencies between threads |
| Replicated data problems | Dependencies between threads can be removed from individual threads |
| Quantization / thresholding | Step 1 of error diffusion; for 8-bit to 1-bit: 0-127 gives 0, 128-255 gives 1 |

## Decomposition comparison

| | Task | Data | Data flow |
|---|---|---|---|
| Idea | Different activities to different threads | Same operation on different data blocks | Output of one thread feeds another |
| Typical use | GUI apps (e.g. typing + pagination) | Audio, imaging, scientific programming | Producer/consumer: file I/O then processing, parsing then code generation |
| Pros / notes | Easiest to program when cleanly partitioned by function | More complex code; use when data is easily divided and performance matters; lets problem size grow | Watch startup/shutdown latency and idle consumers |
| Patterns | Task-level parallelism; Divide and Conquer | Geometric Decomposition; Divide and Conquer | Pipeline; Wavefront |

## Patterns at a glance

| Pattern | Key idea | Examples / notes |
|---|---|---|
| Task-level parallelism | Independent tasks; remove or replicate dependencies | Embarrassingly parallel, replicated data |
| Divide and Conquer | Split into sub-problems, solve independently, aggregate | Merge sort; good load balance and locality |
| Geometric Decomposition | Threads operate on chunks of a data structure | Heat flow, wave propagation |
| Pipeline | Stages like an assembly line; each thread on a different stage | Solves producer/consumer scalably |
| Wavefront | Process elements along diagonals of a 2-D grid | Load balancing is key; error diffusion by rows |

## Numbers and facts to remember

| Fact | Value |
|---|---|
| Error diffusion proposed by | Floyd and Steinberg (1975) |
| Floyd-Steinberg weights | 7/16 right, 3/16 below-left, 5/16 below, 1/16 below-right |
| Threshold (8-bit to 1-bit) | 0-127 gives 0; 128-255 gives 1 |
| Worked example | Input 168, output 1 (255), error = -87 |
| Inputs a pixel needs | 3 errors from previous row (eA, eB, eC) + 1 from left pixel (eD) |
| Row-based startup latency | 2 pixels (vs 10,200 pixels per row at 1,200 dpi, 8.5 x 11 in) |
| Page memory at 1,200 dpi, 8.5 x 11 in | 131 MB |
| Hybrid approach startup latency | Thread 2: 1/3 page; Thread 3: 2/3 page |
| Four challenges | Synchronization, Communication, Load balancing, Scalability |

# PART C: PRACTICE QUESTIONS WITH MODEL ANSWERS

## Short answer (2 to 3 marks)

**Q. What is decomposition?**

Breaking a program down into individual tasks and identifying the dependencies between them. A problem can be decomposed by task, by data, or by data flow.

**Q. Define task decomposition with an example.**

Decomposing a program by the functions it performs, with different activities assigned to different threads. Example: in a word processor, text entry and background pagination are separate tasks running in parallel.

**Q. Define data decomposition. Why is it also called data-level parallelism?**

It breaks tasks down by the data they work on: many threads perform the same operation on different data items (n threads each doing 1/n of the work), e.g. recalculating a large spreadsheet.

**Q. What is data flow decomposition?**

Breaking a problem up by how data flows between tasks, where the output of one thread is the input to another. The producer/consumer problem is the classic example.

**Q. List the four challenges of threading.**

Synchronization, communication, load balancing and scalability.

**Q. What is an embarrassingly parallel problem?**

One in which there are no dependencies between threads (task-level parallelism pattern).

**Q. Name the five parallel programming patterns in Table 3.2 and their decompositions.**

Task-level parallelism (task); Divide and Conquer (task/data); Geometric Decomposition (data); Pipeline (data flow); Wavefront (data flow).

**Q. What is the first step of the error diffusion algorithm?**

Determine the output value for the current pixel by quantization or thresholding: for 8-bit input to a 1-bit device, 0 to 127 gives 0 and 128 to 255 gives 1.

**Q. Why is a two-pixel latency acceptable in row-based parallel error diffusion?**

A page of 8.5 x 11 in at 1,200 dpi has 10,200 pixels per row, so a two-pixel startup latency is insignificant.

## Medium answer (5 marks)

**Q. Compare task, data and data flow decomposition using the gardener analogy.**

Task: one gardener mows while the other weeds, needing coordination. Data: both mow half the property then both weed half the beds; also lets problem size grow as cores are added (two extra gardeners take a second yard). Data flow: one gardener prepares the tools for both; gardening can start only when this is mostly done, then both proceed in parallel.

**Q. Explain the producer/consumer problem and its dimensions.**

Output of one task (producer) is input to another (consumer) in a different thread; consumer cannot start until the producer completes some portion. Examples: file read then processing; parsing then code generation. Dimensions: (1) dependence can cause significant delay, so understand and minimise it and avoid idle consumers; (2) ideal clean hand-off is context-independent, but often interaction needs careful scheduling; (3) if one side finishes while the other is still busy, a thread idles, violating load balancing.

### Q. How do you choose between task and data decomposition?

Task decomposition is usually the winner when the goal is ease of programming and the work partitions cleanly by function. Data decomposition adds code complexity, so it is used when data is easily divided and performance matters. Often the problem domain or a resource constraint decides (e.g. only one mower); otherwise careful planning and empirical timing decide. Example: video frames with no inter-frame dependency can use either; in the gardener case task decomposition won because mowing start-up time is saved.

### Q. Explain the four challenges of threaded programming.

Synchronization: threads coordinate, e.g. one waits for another to finish. Communication: bandwidth and latency of data exchange between threads. Load balancing: roughly equal work per thread. Scalability: efficient use of more threads on a more capable system, e.g. a four-core design running well on eight cores.

### Q. Describe the divide and conquer and geometric decomposition patterns.

Divide and conquer: split the problem into independent sub-problems, solve in parallel, aggregate results; used in merge sort; gives good load balancing and locality. Geometric decomposition: parallelise the data structures, each thread working on a chunk; used for heat flow and wave propagation.

### Q. Explain the wavefront pattern with reference to Figure 3.1.

Elements of a 2-D grid are processed along diagonals; the number on each element gives processing order; diagonal 3 depends on 1 and 2; shaded elements are done. It is a data-flow pattern; minimising thread idle time and load balancing are the keys to success.

### Q. Explain the three steps of the error diffusion algorithm with the 168 example.

(1) Threshold: 168 is at least 128, so the output is 1. (2) Error: normalise the output to 255; error = 168 - 255 = -87. (3) Distribute the error to neighbours using Floyd-Steinberg weights: 7/16 right, 5/16 below, plus 3/16 below-left and 1/16 below-right; repeat for all pixels.

## Long answer (8 to 10 marks)

### Q. Why does error diffusion appear serial, and how can it be parallelised?

The conventional algorithm pushes errors to neighbours as each pixel is computed, so the previous pixel's error must be known before the next can be computed, implying one pixel at a time. Viewed from the receiving pixel (Figure 3.4), each pixel needs three errors from the previous row (eA, eB, eC) and one from the left (eD): multiple producers and one consumer, hence data-flow decomposition. Splitting a row into even and odd pixels fails because each thread blocks on the other and may be slower than serial. Instead, assign one thread per row: once a row has done its first few pixels, the next row's thread can start, so many rows are processed simultaneously (wavefront pattern, Figure 3.5). Startup latency is two pixels per row, insignificant against 10,200 pixels per row for a 1,200 dpi, 8.5 x 11 in page.

### Q. Compare row-based, page-based and hybrid partitioning for parallel error diffusion.

Page-based is the simplest (pages are independent) but restricts to one image per page, uses much more memory (131 MB per page at 1,200 dpi, intermediate results saved) and gives no speedup when printing one page. Hybrid: threads process regions of a page, but each thread works on sections of different pages, increasing startup latency (1/3 page for Thread 2, 2/3 page for Thread 3), so it shares the same limitations. Row-based (Figure 3.5) avoids these: only a two-pixel latency, no one-image-per-page restriction and low memory, so it is the recommended approach.

### Q. Describe the parallel programming patterns and relate each to a decomposition.

Task-level parallelism (task): independent tasks, remove or replicate dependencies; embarrassingly parallel and replicated data problems. Divide and conquer (task/data): sub-problems solved independently then aggregated; e.g. merge sort; good balance and locality. Geometric decomposition (data): threads on chunks of data structures; heat flow, wave propagation. Pipeline (data flow): assembly-line stages with a thread per stage. Wavefront (data flow): diagonal processing in a 2-D grid, load balancing is key.